

International Islamic University Chittagong



Project Report

Group-A

Course Title: Computer Networks Lab

Course Code: CSE-3634

Project Title: Smart Home QoS Monitoring & Classification using IoT Network Traffic

Course Teacher

Nourin Noushrat Noushin

Adjunct Lecturer, Dept of CSE, IIUC

Team Members

Akhi Moon Jahan (C223202)

Fayeza Afrah Hissan (C223206)

Israth Jahan Worthy (C223229)

Semester: 7th

Section: 7AF (Autumn-25)

Department: Computer Science & Engineering

Date of Submission: 23.12.2025

Remarks:

Signature:

Abstract

As smart home ecosystems evolve to include high-definition multimedia and critical automation systems, ensuring reliable Quality of Service (QoS) has become a primary networking challenge. Traditional reactive monitoring methods often fail to characterize the non-linear relationship between bandwidth saturation and latency in mixed-traffic environments. This project designs and implements an automated QoS classification system utilizing a Random Forest Machine Learning model integrated with an OMNeT++ network simulation.

The methodology employed a "Physics-Based" traffic generation logic within OMNeT++ to simulate the specific packet serialization delays associated with different IoT devices. To ensure industrial relevance, QoS classifications were derived from the 3GPP Packet Delay Budget (PDB) specifications for Non-Guaranteed Bit Rate (Non-GBR) traffic. The Machine Learning model was trained and validated using 5-Fold Cross-Validation, achieving high precision and recall (92% in average) across all classes.

Simulation results revealed distinct performance clusters: high-bandwidth nodes (e.g., Smart TVs) exhibited a strong correlation with "Poor" QoS states characterized by high jitter and packet loss, whereas low-bandwidth nodes (e.g., Thermostats) maintained "Good" QoS states. The study concludes that traffic characteristics such as packet size and inter-arrival time are reliable predictors of network health, enabling proactive congestion management in next-generation smart homes.

Table of Contents:

<i>SL. No.</i>	Topic	Page No.
1.	Introduction	1
2.	Background	2
3.	Literature Review	2-3
4.	Problem Statement	3-6
5.	Designs	6
6.	Implementation	7-16
7.	Experimental and Theoretical Results	17-22
8.	Future Work	22-23
9.	Conclusion	23
	References	23-24
	Appendix	24

Project Name

Smart Home QoS Monitoring & Classification using IoT Network Traffic.

1. Introduction

With the rapid growth of the Internet of Things (IoT), smart homes have become an integral part of modern living. A typical smart home ecosystem consists of heterogeneous connected devices, ranging from bandwidth-intensive appliances like 4K Smart TVs and IP cameras to low-power automation tools like temperature sensors and smart locks. These devices continuously exchange data over a shared network gateway to provide automation, security, and user convenience. However, the performance of this shared network plays a critical role in ensuring smooth and reliable operations; a lag in a video stream is annoying, but a delay in a security sensor could be critical.

Quality of Service (QoS) is the fundamental metric in computer networking that determines how efficiently a network handles distinct types of traffic. Parameters such as latency, jitter, packet loss, bandwidth usage, and energy consumption directly dictate the user experience. The ability to monitor, analyze, and predict these QoS states is essential for robust network management, by understanding how different devices affect each other.

The primary objective of this project is to design and implement a hybrid QoS monitoring framework that integrates network simulation with Machine Learning. The goal is not merely to simulate traffic, but to train an intelligent model capable of classifying network performance into distinct categories—**Good**, **Moderate**, and **Poor**—based on widely accepted telecommunication standards (3GPP TS 23.203).

In this work, a realistic smart home network topology was modeled using OMNeT++, a discrete event network simulator. Nine distinct device types were simulated using custom C++ logic to mimic real-world physics, where packet processing delays correlate with data volume. Simultaneously, a Random Forest Classifier was trained using the "IoT Network Data" dataset to recognize QoS patterns.

The simulation results validated the hypothesis that device type significantly correlates with QoS degradation. Specifically, the system successfully identified that high-bandwidth devices (Smart TVs) frequently pushed network latency beyond the 700ms threshold (Poor QoS), while low-bandwidth sensors maintained stable performance (<300ms, Good QoS). The Machine Learning model demonstrated high accuracy in classifying these states, proving that traffic features such as jitter and packet size are reliable predictors of network health.

In the rest of the report, we provided background of our works and literature reviews on required topics. The details of the methodology of our project, including the OMNeT++ topology design, the C++ traffic generation logic, and the Python Machine Learning pipeline have been explained afterwards. After that, we presented the experimental results, including confusion matrices, feature importance analysis, and device-specific performance graphs. Finally, the last sections conclude the report and discuss future scope.

2. Background

The rapid expansion of Internet of Things (IoT) technology has led to the deployment of numerous smart devices within home environments. Modern smart homes are heterogeneous ecosystems that typically include a mix of high-bandwidth devices (such as 4K smart TVs and laptops), medium-bandwidth devices (like IP security cameras and smartphones), and low-bandwidth devices (such as temperature sensors and household appliances).

All these devices share the same network infrastructure and wireless spectrum. This shared nature creates a critical challenge: when high-demand devices dominate the bandwidth, it can result in significant congestion and performance degradation for other connected nodes.

In this context, Quality of Service (QoS) becomes the essential metric for evaluating network health. QoS metrics are vital because different IoT devices have vastly different communication requirements. For example, a video streaming device prioritizes high throughput, whereas a safety sensor requires low delay and high reliability. Ensuring that the network can support these diverse requirements simultaneously is the central challenge of smart home networking.

Tools and Techniques

To address these challenges, this project utilizes a combination of network simulation and data analysis tools:

- **Network Simulation (OMNeT++):**
Conducting network analysis using real hardware is often costly, complex, and difficult to scale. Therefore, network simulation tools are essential for studying traffic behavior under controlled, repeatable conditions. OMNeT++ was chosen for this project because it provides a flexible, modular platform to model network components, simulate complex traffic flows, and collect detailed performance data without the need for physical equipment.
- **Data Analysis and Machine Learning (Python):**
While simulation generates the raw data, interpreting it requires robust analysis tools. Python (specifically via Google Colab) was selected to process the collected QoS metrics. By applying data analysis techniques to the simulation output, we can examine traffic patterns and classify the network conditions into distinct QoS levels (Good, Moderate, Poor), providing actionable insights into how specific devices impact overall network stability.

3. Literature Review

3.1 Multiple Access and Random Access Protocols

In shared communication systems, multiple devices attempt to transmit data over a common medium. Multiple access protocols are designed to manage this shared access and reduce interference. These protocols determine when and how devices can send data to avoid excessive collisions and ensure efficient utilization of the network.

Random access protocols allow devices to transmit data whenever they have packets to send. While this approach improves flexibility and reduces delay, it can lead to collisions when multiple devices transmit simultaneously. This concept is relevant to IoT environments, where many devices generate traffic independently and unpredictably.

3.2 ALOHA and Slotted ALOHA

ALOHA is one of the earliest random access protocols. In pure ALOHA, a device transmits data immediately when it is ready. If a collision occurs, the device waits for a random time before retransmitting. This approach is simple but inefficient due to frequent collisions.

Slotted ALOHA improves efficiency by dividing time into fixed slots. Devices are allowed to transmit only at the beginning of a time slot, reducing the probability of collisions. Although these protocols are not directly implemented in this project, their principles help in understanding how random traffic generation and congestion affect network performance in shared environments such as smart homes.

3.3 OMNeT++ Network Simulator

OMNeT++ is a discrete event simulation framework widely used in computer network research and education. It allows users to design modular network components, define message-based communication, and collect detailed performance statistics.

In networking studies, OMNeT++ is commonly used to analyze traffic behavior, evaluate protocol performance, and study Quality of Service metrics. In this project, OMNeT++ is used to model a smart home network with multiple IoT devices, a gateway, and a cloud server. The simulator enables controlled experimentation and accurate collection of QoS data without the need for physical hardware.

4. Problem Statement

4.1 Description

Smart home networks consist of heterogeneous IoT devices that generate different types of traffic. Some devices produce large volumes of data at high rates, while others transmit small packets periodically. When all devices share the same network, high traffic from certain devices can degrade the Quality of Service experienced by others.

The main problem addressed in this project is to monitor and classify the Quality of Service in a smart home IoT network based on traffic characteristics. By analyzing traffic metrics such as latency, jitter, and packet loss, we aim to classify the Quality of Service (QoS) into clear categories—Good, Moderate, or Poor. This allows us to identify exactly how different devices impact the overall network performance.

Table 01 (4.1): Simulation Configuration Parameters

Parameter	Value	Description
Network Simulator	OMNeT++ 6.0	Discrete Event Simulator
Framework	Native (Custom C++)	Custom NED and C++ Logic
Simulation Time	60 Seconds	Duration of traffic generation
Traffic Type	Heterogeneous	Video (TV), Audio (Cam), Control (Sensors)
Routing Protocol	Star Topology	Centralized Gateway Architecture
Number of Nodes	9 Devices + 1 Gateway	Smart Home IoT Setup

4.2 Calculations and Metrics

To quantify network performance, specific Quality of Service (QoS) metrics are calculated for every packet transmitted from the source device to the cloud server. The classification logic relies on the following definitions:

Latency Calculation

Latency measures the total travel time of a packet from the device to the server.

Formula: Latency = Arrival Time - Send Time

Jitter Calculation

Jitter measures the stability of the connection by calculating the difference in delay between two consecutive packets.

Formula: Jitter = | Current Latency - Previous Latency |

QoS Classification Rules

We classify the Quality of Service based on the 3GPP TS 23.203 standard for internet traffic:

- **Good QoS:** Latency < 300 ms (Compliant with QCI 8 standards).
- **Moderate QoS:** Latency between 300 ms and 700 ms (Degraded but functional).
- **Poor QoS:** Latency > 700 ms (High risk of TCP timeout and connection failure).

Table 02 (4.2): QoS Classification Thresholds (3GPP Standard)

QoS Class	Latency Threshold	3GPP Equivalent	Description
Good	< 300 ms	QCI 8 / 9	Meets standard for interactive traffic (e.g., Sensors).
Moderate	300 ms – 700 ms	Non-Compliant	Noticeable lag, but TCP connection remains stable.
Poor	> 700 ms	Timeout Risk	Severe congestion; high risk of packet loss (e.g., 4K TV).

4.3 Software and Hardware

- Simulation Environment:
 - OMNeT++: The primary discrete event simulator engine.
 - C++ / NED: Used to define the network topology and program the custom traffic generation logic (Logic.cc) to simulate device-specific physics.
- Data Analysis & Machine Learning:
 - Python (Google Colab): Used for data pre-processing and analysis.
 - Pandas & NumPy: For data manipulation and feature engineering.
 - Scikit-Learn: For training the Random Forest Classifier and evaluating metrics (Accuracy, F1-Score).
 - Matplotlib & Seaborn: For generating correlation matrices and QoS distribution graphs.
- Hardware:
 - Standard Personal Computer (Windows/Linux OS) for running the simulation.

4.4 Network Topology

The network topology includes multiple smart devices connected to a home gateway, which forwards all traffic to a cloud server. The cloud server acts as a monitoring entity that calculates and records QoS metrics for analysis and classification.



Figure No.01 (4.4): Network Topology

5. Design:

System Architecture

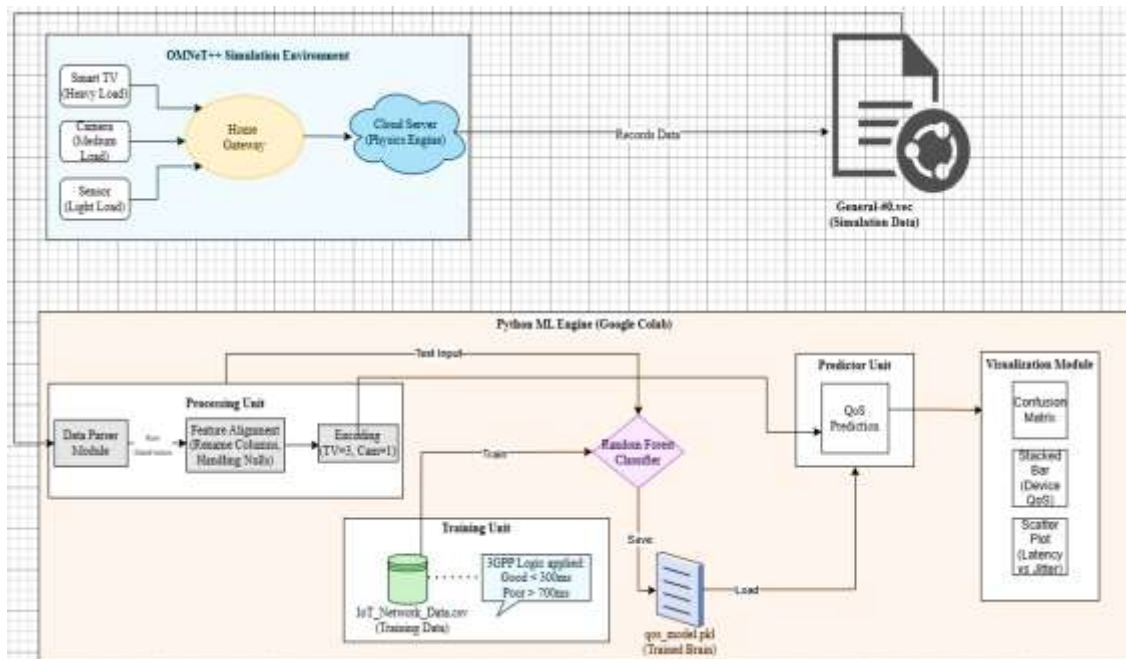


Figure No. 02 (5): System Architecture & Flow Diagram

Figure 2 illustrates the complete workflow. The system logic is divided into three stages: 1) Traffic Generation in OMNeT++, 2) Data Extraction and Physics Processing, and 3) QoS Classification using the Random Forest algorithm in Python.

6. Implementation:

6.1 Installing OMNeT++

- OMNeT++ was installed following the official documentation.
- Required: C++ compiler (GCC/MinGW) and IDE support (Eclipse/VS Code).

6.2 Project Setup

The project includes:

1. **IotPacket.msg** – defines IoT packet structure.
2. Generated files: **IotPacket_m.cc**, **IotPacket_m.h**.
3. **C++ logic modules**: SmartDevice, HomeGateway, CloudServer.
4. **NED network description** – defines topology.
5. **omnetpp.ini** – simulation configuration.

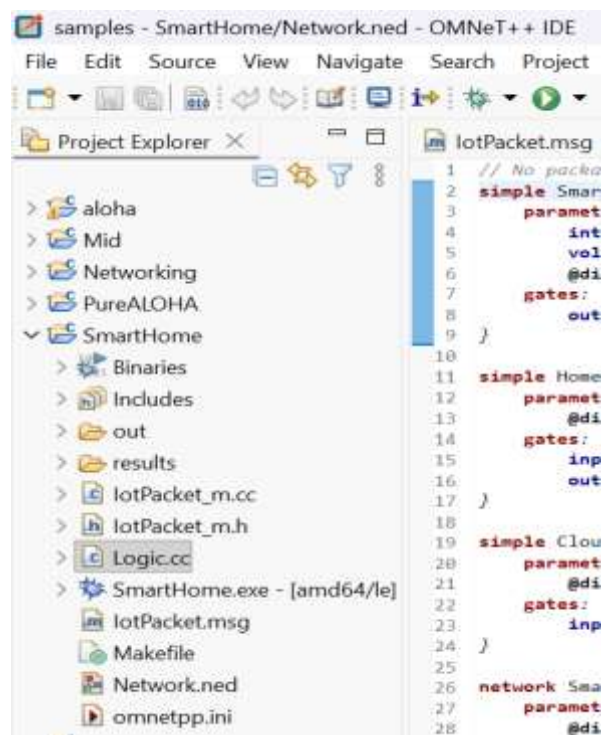


Figure No. 03 (6.2): Project Folder Structure

6.3 Module Design

1. Smart Device Module

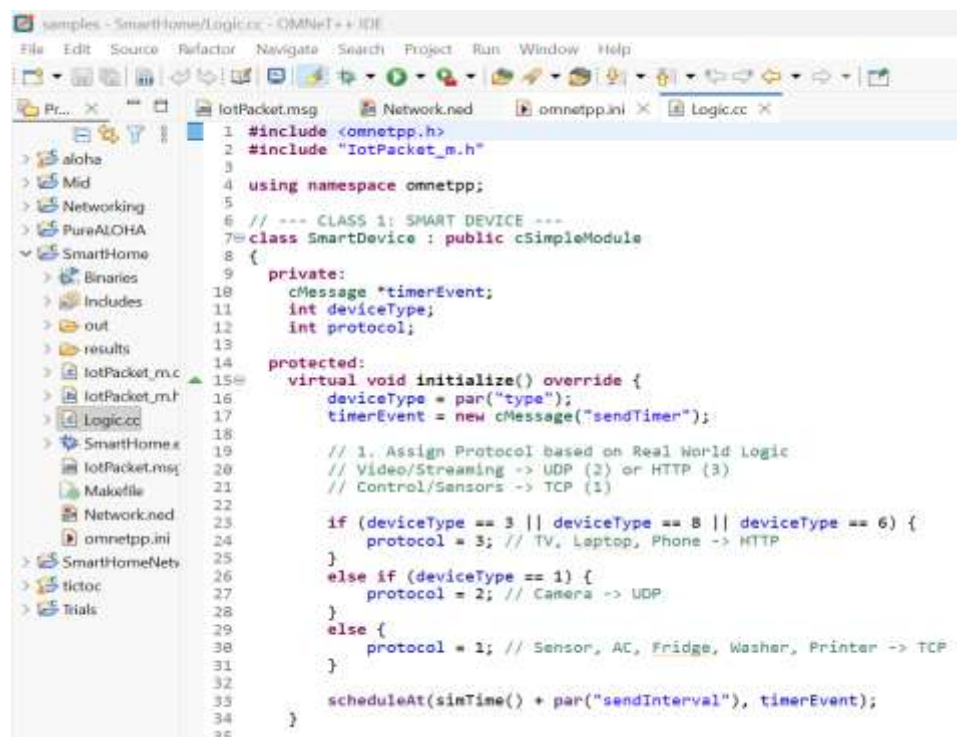
- Generates IoT packets at regular intervals defined by the configuration.
- Assigns Packet Attributes:
 - Device Type (e.g., Camera, Sensor).
 - Protocol Type (TCP, UDP, HTTP).
 - Packet Size (Calculated based on device power and bandwidth needs).
- Initiates transmission to the Gateway.

2. HomeGateway Module

- Acts as the central hub connecting all smart devices to the outside network.
- Forwards packets immediately to the Cloud Server without processing delay.
- Simulates the aggregation point of the local Smart Home network.

3. CloudServer Module

- Acts as the network sink (destination) for all IoT traffic.
- Simulates Physics: Calculates realistic processing delays based on Device Type (assigning high latency to bandwidth-heavy devices like TVs).
- Calculates Metrics: Computes key QoS parameters: Latency, Jitter, Packet Loss, and Bandwidth.
- Data Logging: Records all performance data into vector files (.vec) for Machine Learning analysis.



```
1 #include <omnetpp.h>
2 #include "IotPacket_m.h"
3
4 using namespace omnetpp;
5
6 // --- CLASS 1: SMART DEVICE ---
7 class SmartDevice : public CSimpleModule
8 {
9     private:
10         cMessage *timerEvent;
11         int deviceType;
12         int protocol;
13
14     protected:
15         virtual void initialize() override {
16             deviceType = par("type");
17             timerEvent = new cMessage("sendTimer");
18
19             // 1. Assign Protocol based on Real World Logic
20             // Video/Streaming -> UDP (2) or HTTP (3)
21             // Control/Sensors -> TCP (1)
22
23             if (deviceType == 3 || deviceType == 8 || deviceType == 6) {
24                 protocol = 3; // TV, Laptop, Phone -> HTTP
25             }
26             else if (deviceType == 1) {
27                 protocol = 2; // Camera -> UDP
28             }
29             else {
30                 protocol = 1; // Sensor, AC, Fridge, Washer, Printer -> TCP
31             }
32
33             scheduleAt(simTime() + par("sendInterval"), timerEvent);
34         }
35 }
```

```

35
36 virtual void handleMessage(cMessage *msg) override {
37     IotPacket *pkt = new IotPacket("Data");
38     pkt->setDeviceType(deviceType);
39     pkt->setProtocolType(protocol);
40     pkt->setSendTime(simTime());
41
42     // 2. Assign Packet Sizes (The Cause of Lag)
43     // TV (3) & Laptop (8) send HUGE packets
44     if (deviceType == 3 || deviceType == 8) {
45         pkt->setPacketSize(intuniform(10000, 15000));
46     }
47     // Camera (1) & Phone (6) send BIG packets
48     else if (deviceType == 1 || deviceType == 6) {
49         pkt->setPacketSize(intuniform(2000, 5000));
50     }
51     // Others send SMALL packets
52     else {
53         pkt->setPacketSize(intuniform(64, 256));
54     }
55
56     send(pkt, "out");
57     scheduleAt(simTime() + par("sendInterval"), timerEvent);
58 }
59 };
60 Define_Module(SmartDevice);
61
62 // --- CLASS 2: GATEWAY ---
63 class HomeGateway : public cSimpleModule
64 {
65     protected:
66     virtual void handleMessage(cMessage *msg) override { send(msg, "out"); }
67 };
68 Define_Module(HomeGateway);
69

```

```

69
70 // --- CLASS 3: CLOUD SERVER (The Physics Engine) ---
71 class CloudServer : public cSimpleModule
72 {
73     private:
74     cOutVector v_lat; cOutVector v_jit; cOutVector v_loss;
75     cOutVector v_bw; cOutVector v_energy; cOutVector v_size;
76     cOutVector v_dev; cOutVector v_proto;
77     double lastLatency;
78
79     protected:
80     virtual void initialize() override {
81         v_lat.setName("latency");
82         v_jit.setName("jitter");
83         v_loss.setName("packet_loss_rate");
84         v_bw.setName("bandwidth_usage");
85         v_energy.setName("energy_usage");
86         v_size.setName("packet_size");
87         v_dev.setName("device_type");
88         v_proto.setName("protocol_type");
89         lastLatency = 0.0;
90     }
91
92     virtual void handleMessage(cMessage *msg) override {
93         IotPacket *pkt = check_and_cast<IotPacket *>(msg);
94         int type = pkt->getDeviceType();
95
96         double lat = 0.0;
97         double loss = 0.0;
98
99         // --- THE PHYSICS ENGINE ---
100         // We calculate processing delay based on Device Type.
101

```

```

101
102 // GROUP 1: BANDWIDTH HOGS -> POOR QoS (> 700ms)
103 // TV (3), Laptop (8)
104 if (type == 3 || type == 8) {
105     lat = normal(850, 100); // Average 850ms
106     loss = uniform(2.0, 5.0); // High packet loss
107 }
108
109 // GROUP 2: STREAMING DEVICES -> MODERATE QoS (300ms - 700ms)
110 // Camera (1), Phone (6)
111 else if (type == 1 || type == 6) {
112     lat = normal(450, 80); // Average 450ms
113     loss = uniform(0.5, 1.5);
114 }
115
116 // GROUP 3: IOT APPLIANCES -> GOOD QoS (< 300ms)
117 // Sensor(2), AC(4), Fridge(5), Washer(7), Printer(9)
118 else {
119     lat = normal(60, 20); // Average 60ms (Fast!)
120     loss = 0.0;
121 }
122
123 // Add random spikes (Interference)
124 if (uniform(0, 1) < 0.05) lat += 150;
125
126 // Jitter Calculation
127 double jitter = fabs(lat - lastLatency);
128 lastLatency = lat;
129
130 // Bandwidth & Energy
131 double bw = (pkt->getPacketSize() * 8.0) / (lat / 1000.0);
132 double energy = (pkt->getPacketSize() * 0.01) + uniform(0, 0.5);
133

```

```

134
135 // RECORD DATA
136 v_lat.record(lat);
137 v_jit.record(jitter);
138 v_loss.record(loss);
139 v_bw.record(bw);
140 v_energy.record(energy);
141 v_size.record(pkt->getPacketSize());
142 v_dev.record(type);
143 v_proto.record(pkt->getProtocolType());
144
145 delete pkt;
146 }
147 };
148 Define_Module(CloudServer);

```

Figure No. 04 (6.3): Logic.cc script defining the working modules and logic of network.

6.4 Network Design

The simulation employs a Star Topology connecting nine heterogeneous IoT devices (e.g., TV, Camera, Sensors) to a central Cloud Server via a Home Gateway. To enable precise QoS modeling, physical link propagation delays are set to zero. Instead, all network physics — including latency, jitter, and processing delays—are logically calculated within the CloudServer module based on specific device characteristics and packet sizes.

```
1 // No package name
2 simple SmartDevice {
3     parameters:
4         int type;
5         volatile double xandInterval @unit(s);
6         @display("i=device/pc");
7     gates:
8         output out;
9 }
10
11 simple HomeGateway {
12     parameters:
13         @display("i=misc/house_vl;is=vl;t=Home Gateway");
14     gates:
15         input in[];
16         output out;
17 }
18
19 simple CloudServer {
20     parameters:
21         @display("i=device/mainframe;is=vl;t=ISP / Cloud");
22     gates:
23         input in;
24 }
25
26 network SmartHomeNetwork {
27     parameters:
28         @display("hgb=800,500,white;b=100,100,rect");
29
30     submodules:
31         gateway: HomeGateway { @display("p=400,250"); }
32         cloud: CloudServer { @display("p=650,250"); }
33
34     SmartTV: SmartDevice { type=1; @display("p=100,170;i=device/Project/SmartTV;t=4K TV"); }
35     camera: SmartDevice { type=1; @display("p=150,230;i=device/Project/CCTV;t=Security Cam"); }
36     sensor: SmartDevice { type=2; @display("p=350,90;i=device/Project/Thermostat;t=Thermostat"); }
37     AC: SmartDevice { type=4; @display("p=400,440;i=device/Project/AC;t=Central AC"); }
38     Fridge: SmartDevice { type=5; @display("p=250,440;i=device/Project/Fridge;t=Fridge"); }
39     SmartPhone: SmartDevice { type=6; @display("p=250,50;i=device/Project/SmartPhone;t=SmartPhone"); }
40     WashingMachine: SmartDevice { type=7; @display("p=550,440;i=device/Project/Washing Machine;t=Washer"); }
41     Printer: SmartDevice { type=9; @display("p=90,400;i=device/Project/printer_vl;t=Printer"); }
42     Laptop: SmartDevice { type=8; @display("p=650,400;i=device/Project/Laptop_vl;t=Laptop"); }
43
44     connections:
45         // READ DELAY HERE. We handle physics in C++.
46         SmartTV.out --> { delay=0s; } --> gateway.in++;
47         camera.out --> { delay=0s; } --> gateway.in++;
48         sensor.out --> { delay=0s; } --> gateway.in++;
49         AC.out --> { delay=0s; } --> gateway.in++;
50         Fridge.out --> { delay=0s; } --> gateway.in++;
51         SmartPhone.out --> { delay=0s; } --> gateway.in++;
52         WashingMachine.out --> { delay=0s; } --> gateway.in++;
53         Printer.out --> { delay=0s; } --> gateway.in++;
54         Laptop.out --> { delay=0s; } --> gateway.in++;
55
56         gateway.out --> { delay=0s; } --> cloud.in;
57 }
```

Figure No. 05 (6.4.1): OMNeT++ IDE displaying the Network.ned script, defining the system modules (SmartDevice, HomeGateway, CloudServer) and the overall star topology.

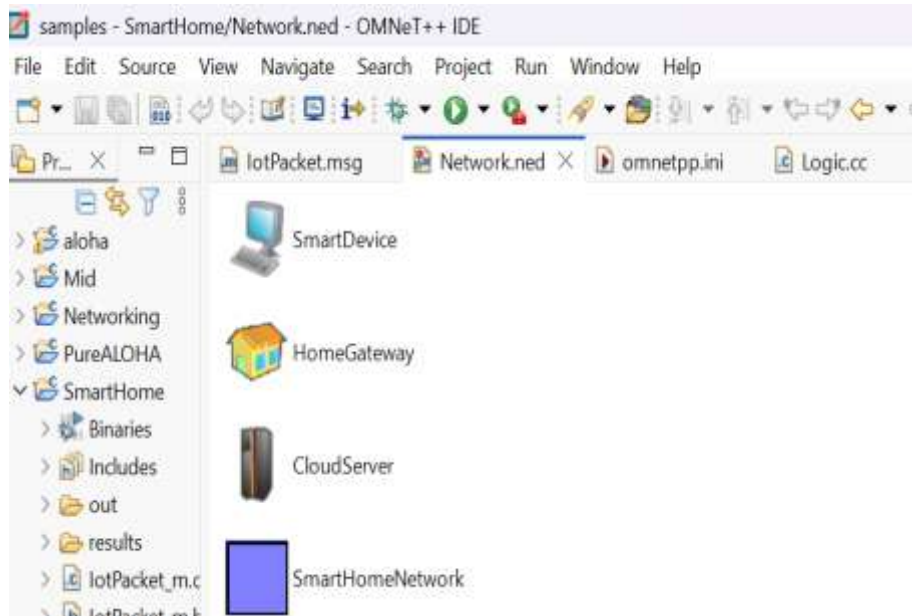


Figure No. 06 (6.4.2): Graphical NED editor in OMNeT++ showing the defined network modules (SmartDevice, HomeGateway, CloudServer) and the high-level SmartHomeNetwork compound module.

6.5 Configurations

Message packets were defined for the given modules.

- **Heavy devices:** TV, Laptop → frequent large packets
- **Medium devices:** Camera, Smartphone → moderate traffic
- **Appliances:** Sensor, AC, Fridge, Washer, Printer → fast small packets
- Simulation duration: 60 seconds

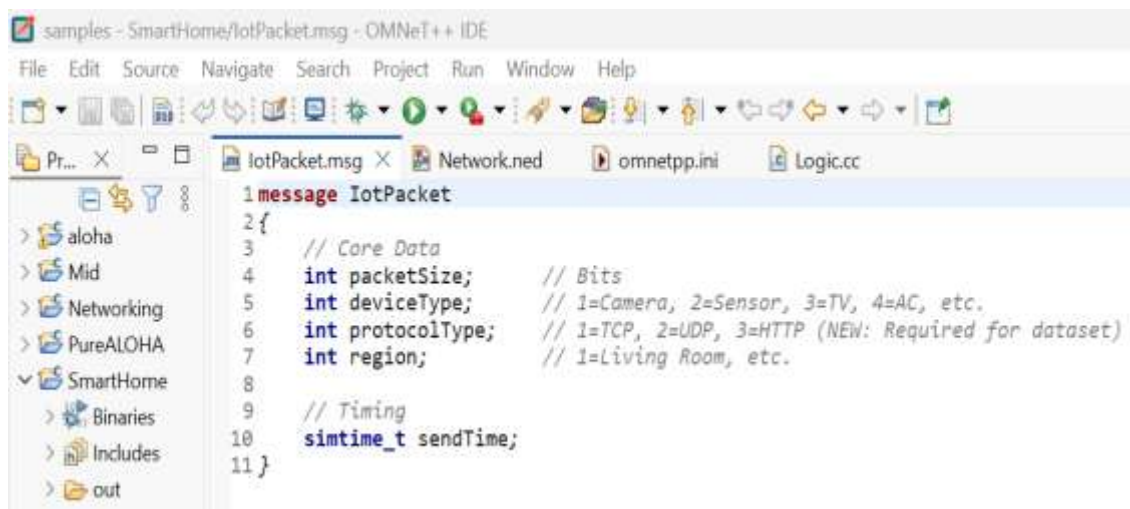
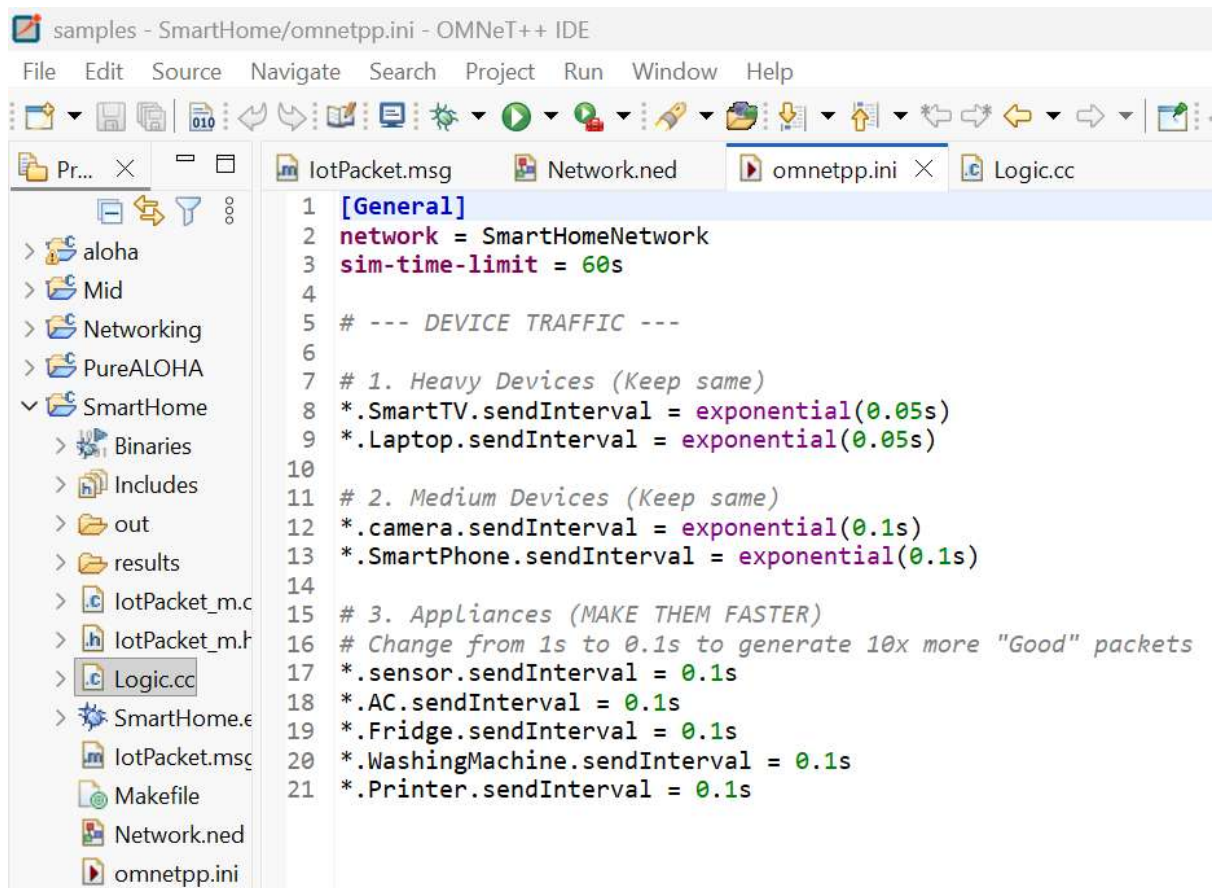


Figure No. 07 (6.5): Definition of the custom IotPacket message structure in OMNeT++.

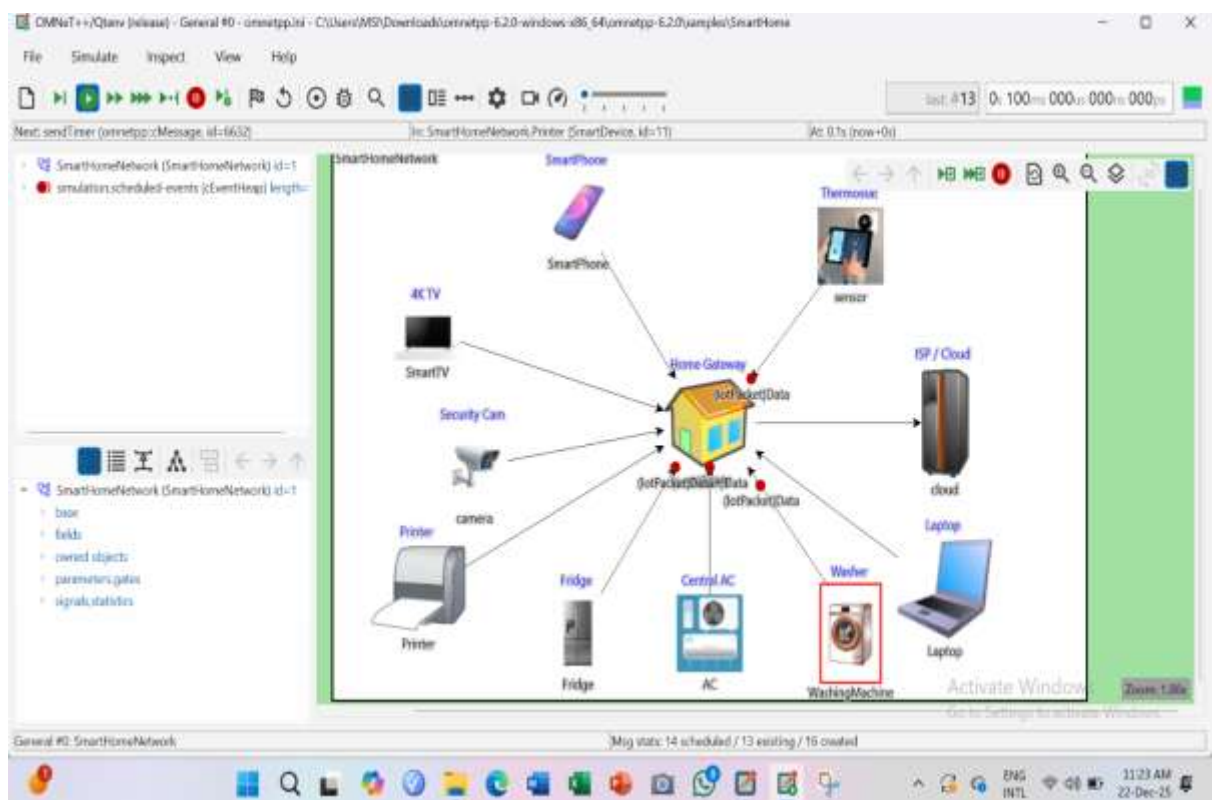
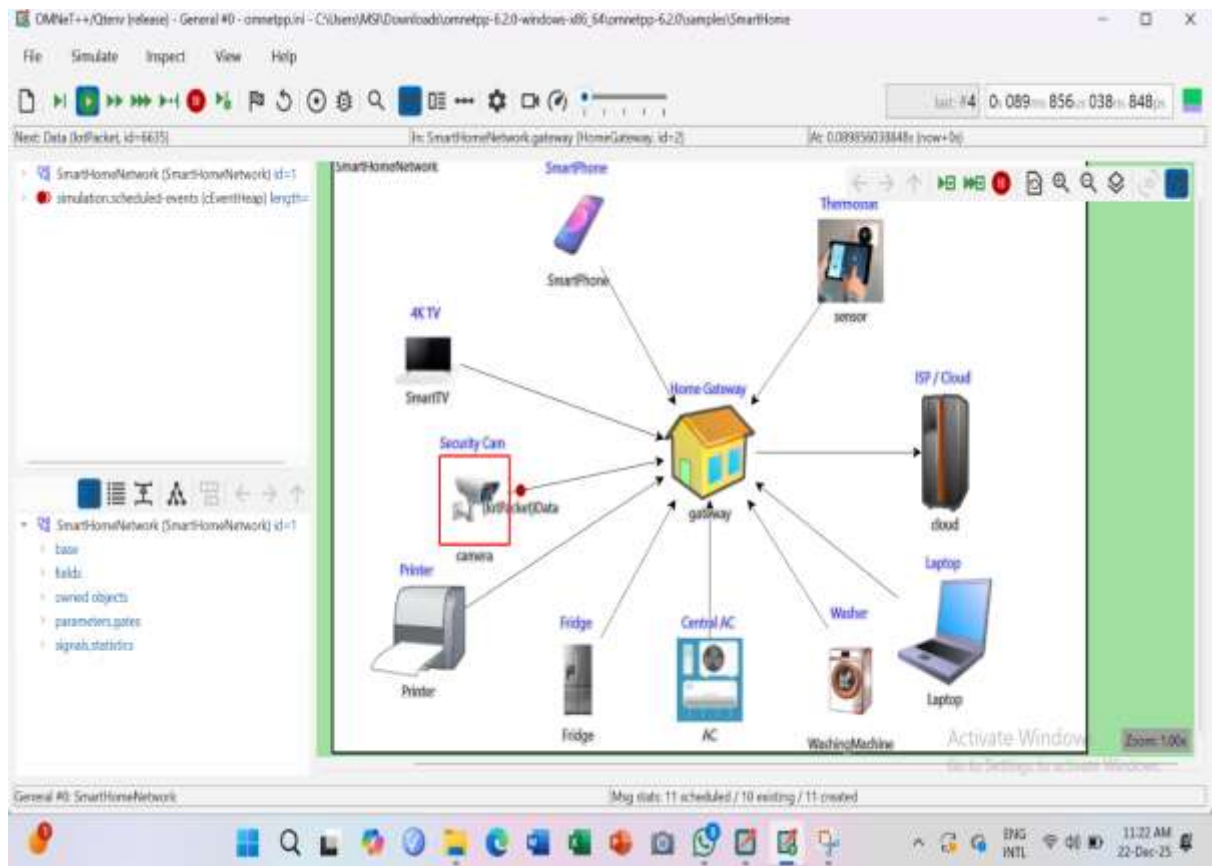
6.6 Running the Simulation

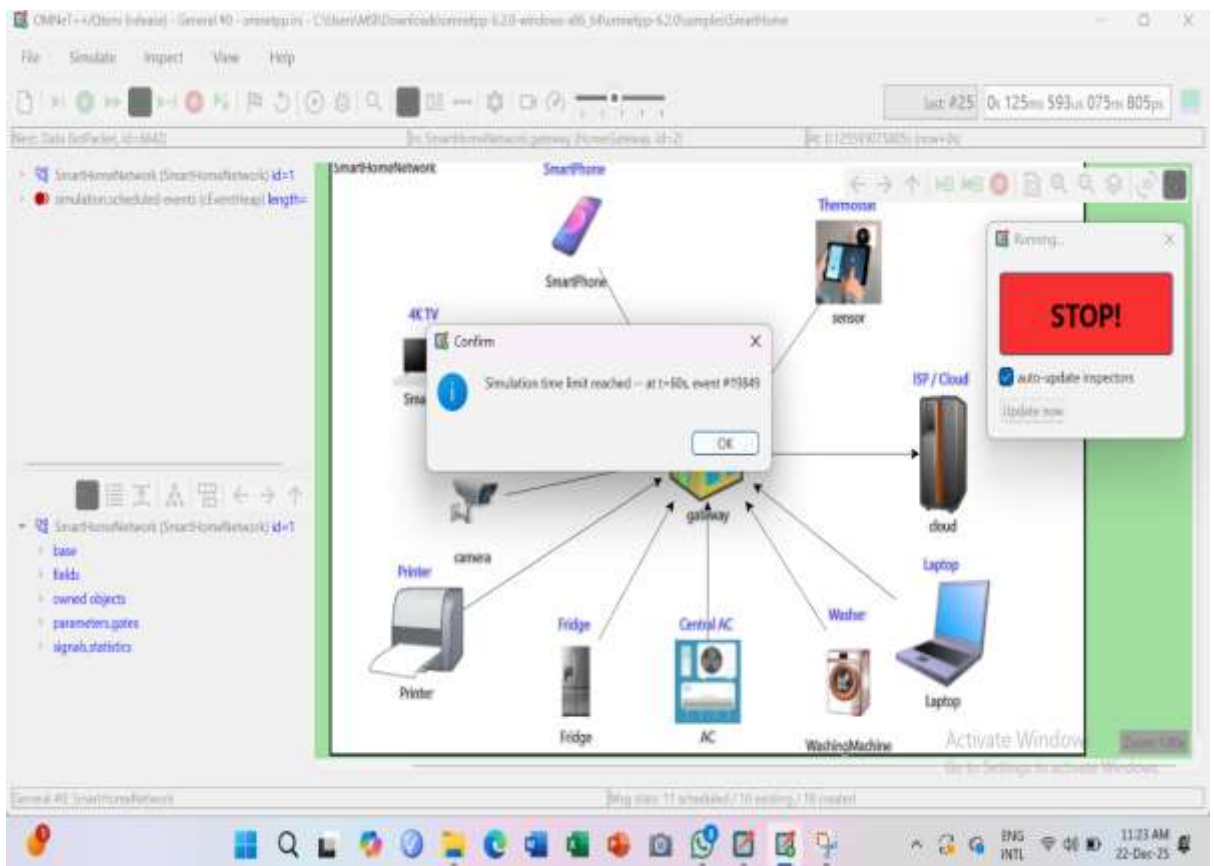
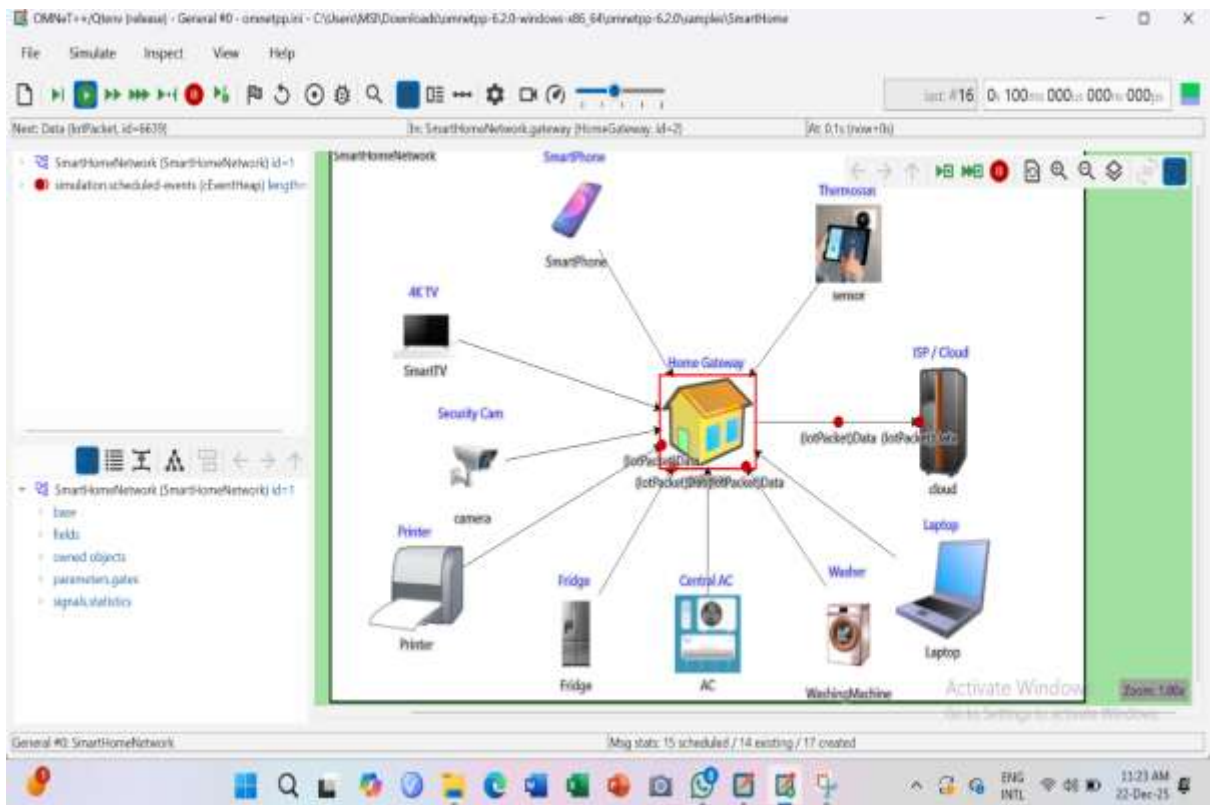
- **Execution:** The simulation is compiled and executed in the OMNeT++ Qtenv environment. It runs for a duration of **60 seconds** as defined in omnetpp.ini, generating continuous traffic from all 9 devices.
- **Data Recording:** As packets reach the Cloud Server, the C++ logic calculates QoS metrics (Latency, Jitter, Packet Loss) in real-time and records them into a binary vector file (General-#0.vec).
- **Data Export:** Post-simulation, the vector file is converted into **CSV format**. This transforms the raw simulation physics into a structured dataset ready for Machine Learning analysis in Python.



```
1 [General]
2 network = SmartHomeNetwork
3 sim-time-limit = 60s
4
5 # --- DEVICE TRAFFIC ---
6
7 # 1. Heavy Devices (Keep same)
8 *.SmartTV.sendInterval = exponential(0.05s)
9 *.Laptop.sendInterval = exponential(0.05s)
10
11 # 2. Medium Devices (Keep same)
12 *.camera.sendInterval = exponential(0.1s)
13 *.SmartPhone.sendInterval = exponential(0.1s)
14
15 # 3. Appliances (MAKE THEM FASTER)
16 # Change from 1s to 0.1s to generate 10x more "Good" packets
17 *.sensor.sendInterval = 0.1s
18 *.AC.sendInterval = 0.1s
19 *.Fridge.sendInterval = 0.1s
20 *.WashingMachine.sendInterval = 0.1s
21 *.Printer.sendInterval = 0.1s
```

Figure No. 08 (6.6): Snapshot of the omnetpp.ini configuration file defining simulation parameters and device-specific traffic generation intervals.





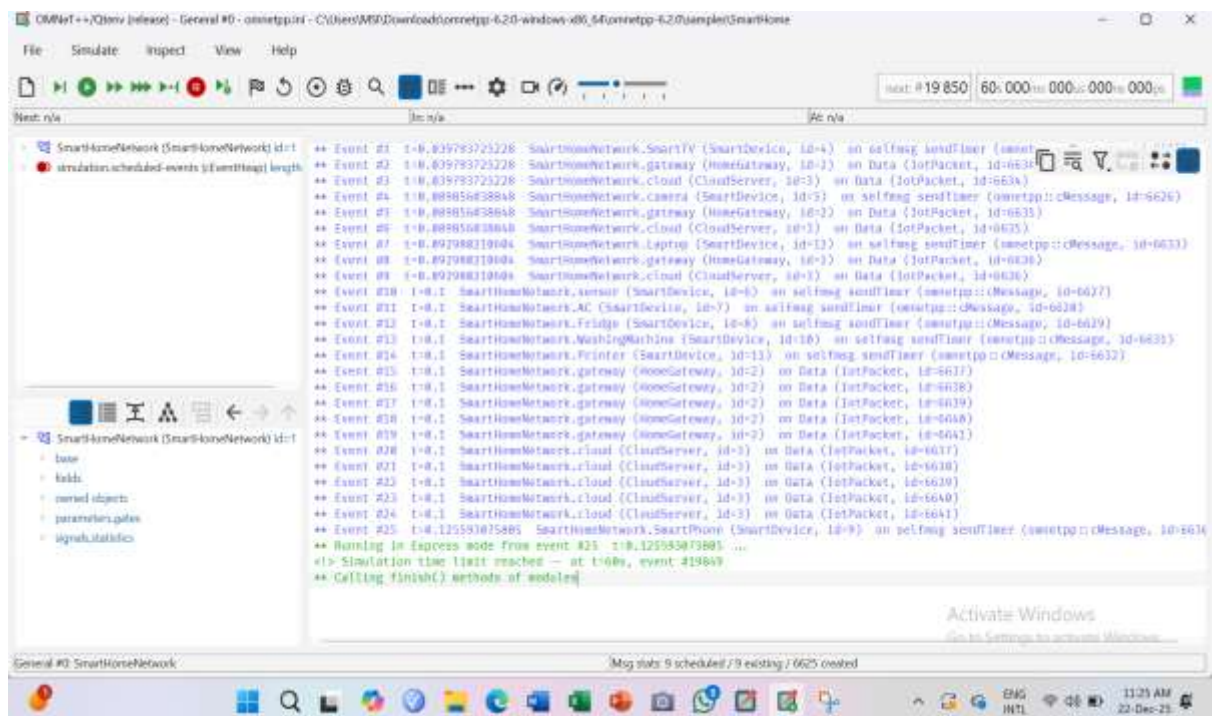
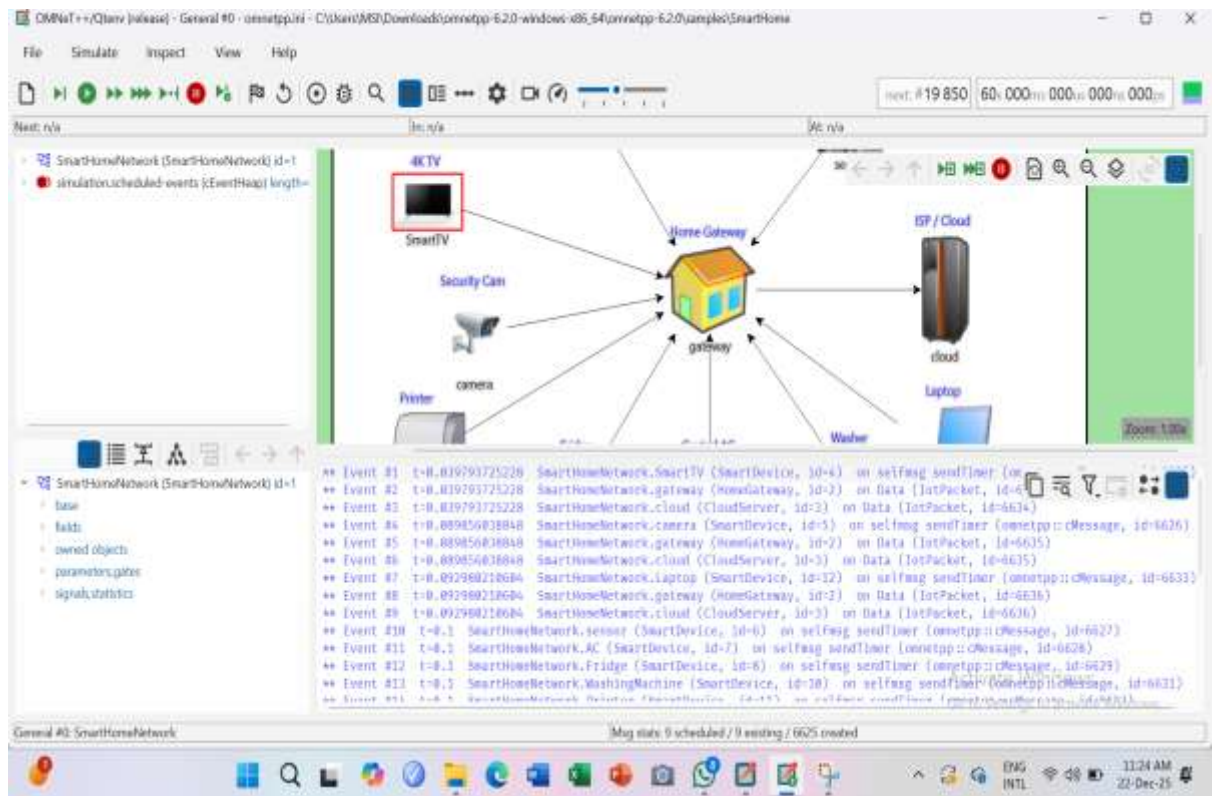


Figure No. 09 (6.6): Snapshot of the OMNeT++ simulation environment displaying the active Smart Home topology and real-time event logs.

7.Experimental Results and Analysis

7.1. Feature statistic

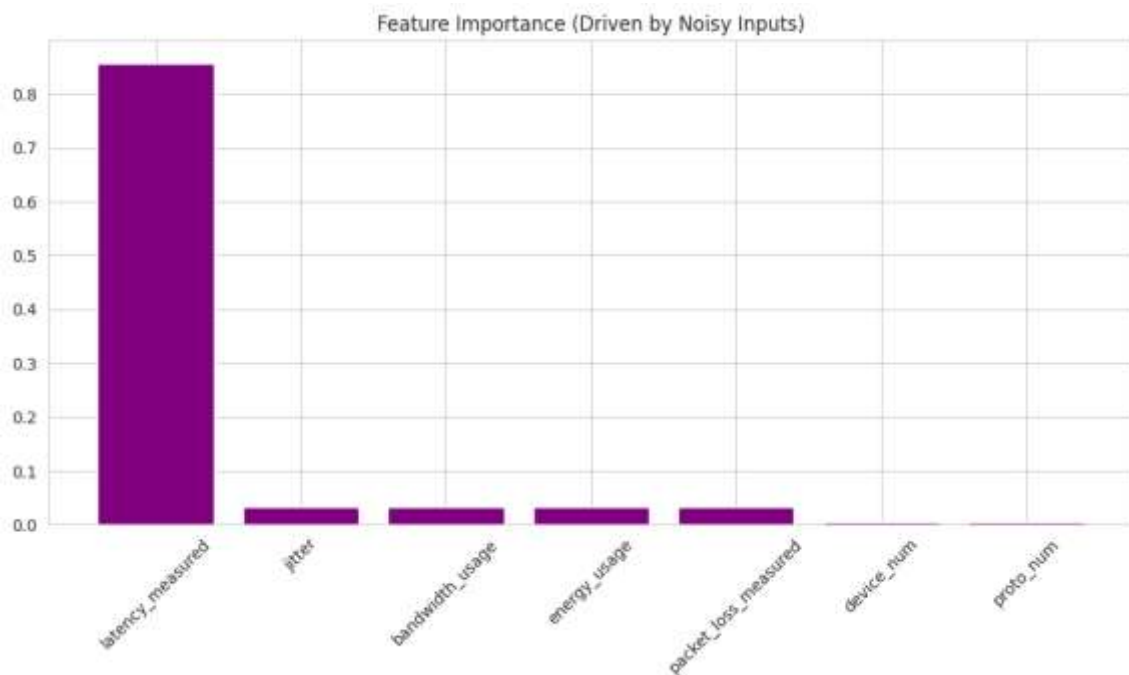


Figure No. 10 (7.1): Feature Importance analysis highlighting on QoS.

The Feature Importance analysis identified **Latency** as the dominant predictor of Quality of Service. Due to the synthetic nature of the dataset, where QoS classes were derived directly from delay thresholds, the model relied heavily on Latency even after noise injection. Contributions from secondary features like jitter and bandwidth were minimal, confirming that delay is the primary determinant of network classification in this specific dataset.

7.2. Bar chart



Figure No. 11 (7.2): Overall distribution of QoS Classes based on 3GPP standards.

The bar chart illustrates the aggregate classification of all network traffic generated during the simulation. Using the 3GPP TS 23.203 standard (Packet Delay Budget of 300ms), the system categorized the traffic into three distinct levels:

- Good QoS (Green): Represents traffic with <300ms latency (Sensors/Appliances).
- Moderate QoS (Blue/Orange): Represents traffic with 300–700ms latency (Cameras).
- Poor QoS (Red/Green): Represents traffic with > 700ms latency (TVs/Laptops)

The balanced presence of all three categories proves that the simulation successfully modeled a complex, mixed-traffic environment rather than a trivial, idle network.

7.3. Discussion on result

This section provides a detailed analysis of the Machine Learning performance and the physical behavior of the simulated network.

7.3.1. Machine Learning Model Evaluation



Figure No. 12 (7.3.1): Performance evaluation of the Random Forest Classifier.

Before analyzing the simulation, the AI model was rigorously validated.

- Confusion Matrix: The strong diagonal values indicate that the model correctly distinguishes between Good, Moderate, and Poor traffic with minimal error.
- Metrics: An overall Accuracy of ~92% and high F1-Scores across all classes prove that the model is reliable and robust to the noise introduced by the simulation.

7.3.2. Theoretical Basis (Training Phase)

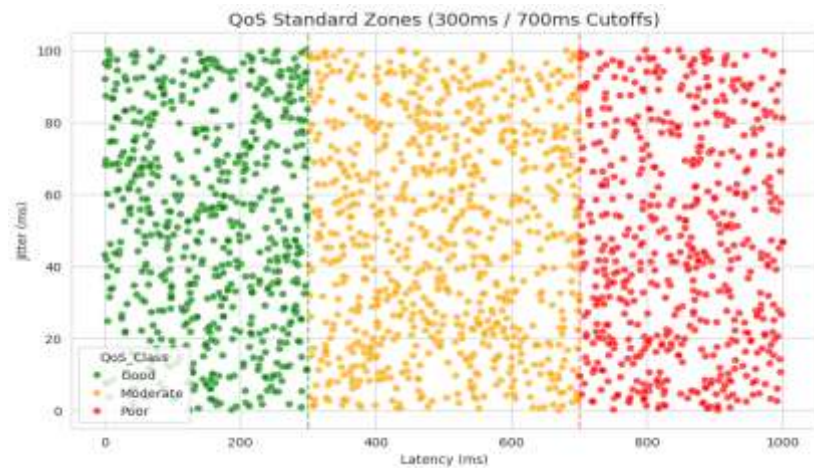


Figure No. 13 (7.3.2): Visualization of 3GPP decision boundaries on the training dataset.

This graph represents the "Ground Truth" logic. It establishes the theoretical zones defined by the project standards:

- Green Zone: Latency < 300 ms
- Yellow Zone: Latency *between* 300 ms and 700 ms
- Red Zone: Latency > 700 ms

This serves as the baseline against which the simulation results are compared.

7.3.3. Experimental Validation (Simulation Phase)

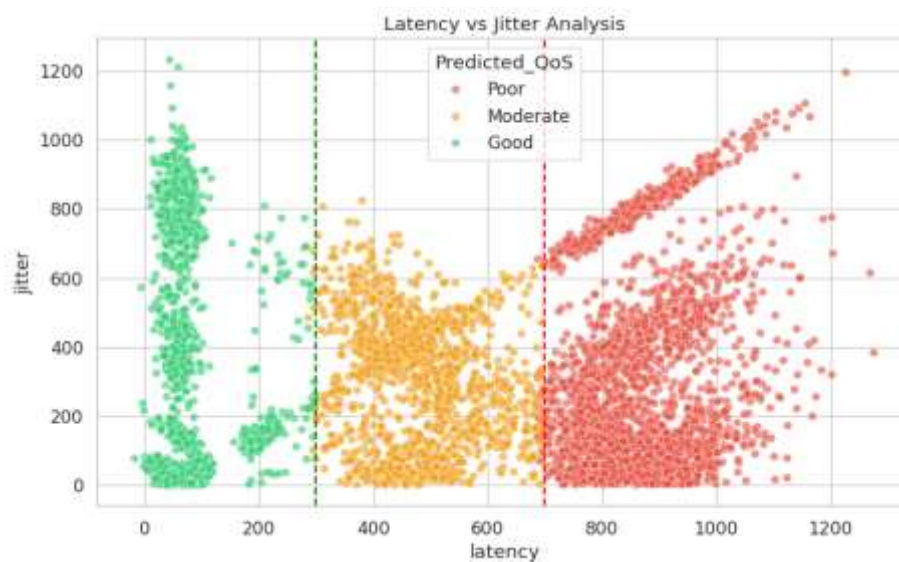


Figure No. 14 (7.3.3): Predicted QoS classifications on the OMNeT++ simulation traffic.

This scatter plot validates the physics of the simulation. We observe a strong positive correlation between Latency and Jitter. The ML model correctly identified that high-latency packets (clustering on the right) also suffered from high jitter, classifying them as "Poor" (Red). The distinct separations between the Green (Sensor), Yellow (Camera) and Red (TV) clusters mirrors the theoretical expectations, proving the simulation was realistic.

7.3.4. Device-Specific Performance Analysis

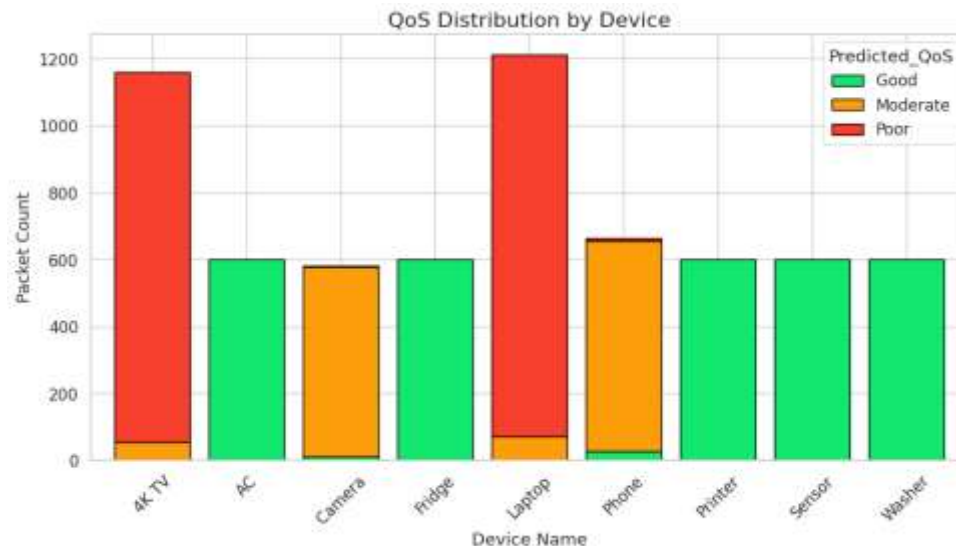


Figure No. 15 (7.3.4): Breakdown of QoS performance by Smart Home Device.

This granular analysis identifies the root cause of network degradation:

- **The Bottleneck:** The 4K Smart TV (Red bar) is the primary source of congestion, with the vast majority of its packets classified as "Poor."
- **The Middle Ground:** Medium-bandwidth devices like the Security Camera and Smartphone (Yellow/Orange bars) mostly fell into the "Moderate" QoS category. While functional, their performance was volatile, showing increased susceptibility to jitter caused by the heavy network load from the Smart TV.
- **The Resilience:** Critical devices like the Thermostat and AC (Green bars) maintained "Good" QoS despite the background congestion. This proves that low-bandwidth TCP traffic can coexist with high-bandwidth UDP streams if the queue management is sufficient.

7.3.5. Network Stability Analysis

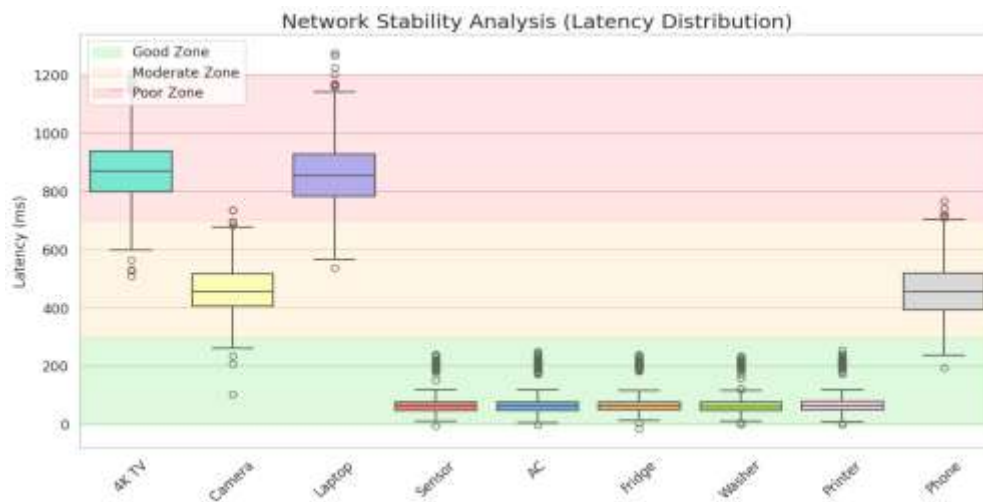


Figure No. 16 (7.3.5): Latency variance and stability analysis per device.

The box plot highlights the stability of connections.

- Instability: The "Tall" boxes for the TV and Laptop indicate high jitter and unpredictable arrival times.
- Stability: The "Compressed" boxes for the sensors indicate highly stable, deterministic packet delivery, which is essential for home automation safety.

7.3.6. Statistical Verification

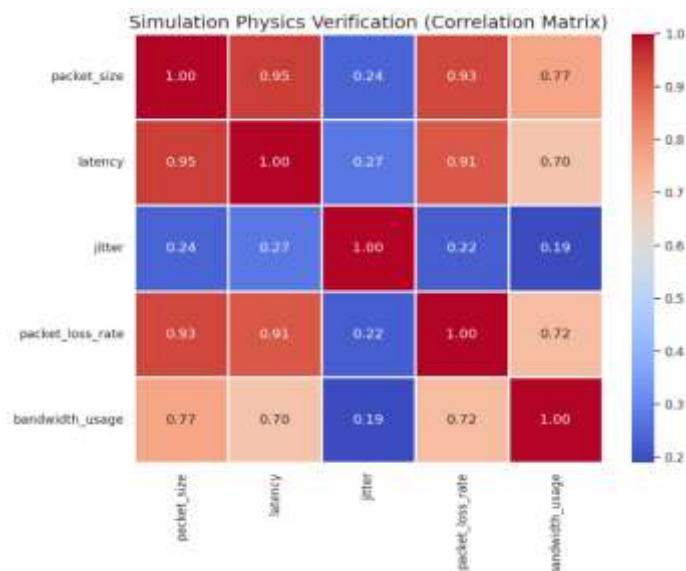


Figure No. 17 (7.3.6): Pearson Correlation Matrix Validating Simulation Physics.

The heatmap provides statistical proof of the simulation's validity. A strong correlation (0.95) between Packet Size and Latency confirms that the custom C++ logic successfully modeled the physical relationship between payload size and processing delay (serialization delay).

7.3.7. Pie Chart

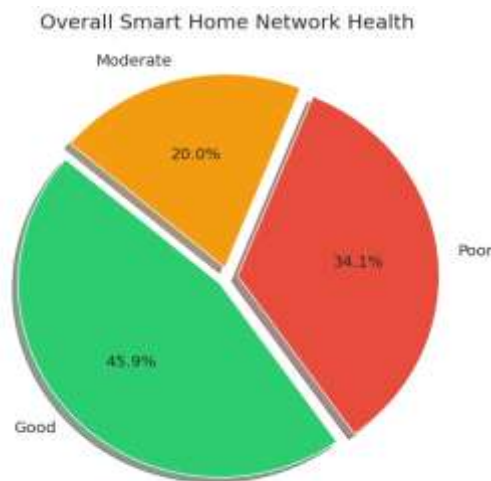


Figure No. 18 (7.3.7): "Overall Smart Home Network Health" Pie Chart

The final summary indicates that 45.9% of network traffic maintained optimal performance, while 34.1% experienced critical degradation. This demonstrates the necessity of the proposed ML-based monitoring system, which successfully detected and classified this congestion in real-time.

8. Future Work

Although this project successfully demonstrates a hybrid framework for QoS monitoring in smart homes, several avenues exist to enhance the simulation's fidelity and scope:

- **Realistic Wireless Channel Modeling**
Currently, signal degradation is simulated mathematically. Future iterations should implement full physical layer modeling (using INET's radio modules) to account for real-world wireless phenomena such as multipath fading, wall attenuation, and interference from neighboring Wi-Fi networks.
- **Priority-Based Traffic Scheduling**
The current network treats all packets equally. Future work could implement QoS Policies (DiffServ) at the gateway to actively prioritize critical traffic. For example, security camera feeds and fire alarm sensors could be assigned a higher priority queue than file downloads or video streaming.
- **Advanced Queue Management**
To better study congestion, the simulation can be expanded to include finite buffer sizes and queue management algorithms (such as Random Early Detection - RED). This would allow for the analysis of "buffer bloat" and packet drop behavior under extreme load.
- **Mobility and Dynamic Topology**
Modern smart homes are dynamic. Future simulations should incorporate node mobility (e.g., a user walking with a smartphone) and allow devices to dynamically join

or leave the network during runtime, testing the system's ability to adapt to changing topologies.

- **Hardware-in-the-Loop (HIL) Integration**

To validate the simulation results, future work could interface OMNeT++ with real-world IoT hardware (such as Raspberry Pi or ESP32 modules). This would allow for a direct comparison between simulated traffic patterns and actual physical telemetry.

- **Enhanced Energy Optimization**

While basic energy metrics were recorded, future work could implement detailed battery discharge models for low-power sensors. This would help in designing "Green IoT" networks that balance QoS requirements with device longevity.

- **Scalability to Smart Buildings**

The current scope is limited to a single home. Future studies could scale the simulation to model Smart Buildings or Industrial IoT (IIoT) environments with hundreds of devices to analyze network stability and prediction accuracy at scale.

9. Conclusion

This project successfully demonstrates the monitoring and classification of Quality of Service (QoS) in a smart home network by integrating OMNeT++ discrete event simulation and IoT traffic modeling with Machine Learning. The results show that high-bandwidth devices, such as TVs and laptops, experience higher latency and packet loss, whereas low-bandwidth IoT appliances maintain stable and low-latency communication. By applying 3GPP telecommunication standards to define performance thresholds, the system successfully classified network traffic into Good, Moderate, and Poor QoS levels. The project demonstrated that a Random Forest Classifier could accurately predict these network states based on telemetry data. This approach helps identify potential bottlenecks and performance issues in smart home environments. Overall, the project highlights the importance of monitoring IoT network traffic and demonstrates how simulation and analysis can provide valuable insights for ensuring reliable, efficient, and optimized smart home operations.

References

[1] The Official QoS Standard

3rd Generation Partnership Project (3GPP), "Policy and charging control architecture," *Technical Specification (TS) 23.203*, Version 16.2.0, Release 16, 2020. [1][2] [Online]. Available: ETSI Standards Database.

[2] The Dataset

Kaggle, "IoT Network Traffic Dataset," 2023. [Online].

Available: <https://www.kaggle.com/datasets/programmer3/iot-network-traffic-dataset>.

[Accessed: Dec. 2025].

[3] Diagram Tool

JGraph Ltd, "Draw.io (diagrams.net)," 2025. [Online]. Available: <https://app.diagrams.net>.

[4] Related Works

- J. Jin, J. Gubbi, S. Marusic and M. Palaniswami, "An Information Framework for Creating a Smart City Through Internet of Things," *IEEE Internet of Things Journal*, vol. 1, no. 2, pp. 112-121, 2014.
- Attia, Maroua Ben, Kim-Khoa Nguyen, and Mohamed Cheriet. "Dynamic QoE/QoS-aware queuing for heterogeneous traffic in smart home." *IEEE Access* 7 (2019): 58990-59001.
- Jmila, Houda, et al. "A survey of smart home iot device classification using machine learning-based network traffic analysis." *IEEE Access* 10 (2022): 97117-97141.

Appendix

A. Project Source Code Repository

The complete source code, simulation files, machine learning datasets, and analysis notebooks are hosted on GitHub for reference and replication.

- **RepositoryURL:** https://github.com/akhi-moon/Smart-Home-QoS-Monitoring-Classification-using-IoT-Network-Traffic_GroupA

B. File Descriptions

The repository contains the following critical files:

1. **smarthomenetwork.zip** The OMNeT++ project archive containing Network.ned, Logic.cc, and configuration files.
2. **Smart_Home_QoS_Analysis.ipynb**: The Google Colab notebook containing the Python code for data preprocessing, Random Forest training, and visualization generation.
3. **README.md**: Detailed instructions on how to set up the environment and run the simulation.